

Advanced Time Series - Computer Exercises 2

Authors

Gunn Jacobsen - s175799
Linda Elisabet á Geilini - s223116

Group: 9

September 18, 2026

Contents

1 Part 1 - Parameter Estimation **1**

2 Part 2 - Contour Plots for the Loss Function **3**

3 Part 3 - Simulation from a Doubly Stochastic System **5**

4 Part 4 - The Exteded Kalman Filter **6**
4.1 Part 4a - Simulating and rewriting the model 6
4.2 Part 4b - Convergence properties of the EKF 7
4.1 Effect of $\sigma_{\mathbf{v}}^{\mathbf{2}}$ 8
4.2 Effect of initial value of parameter a 9
4.3 Effect of the initial value of the variance for parameter a 9

5 References **10**

6 Appendex A **11**

1 Part 1 - Parameter Estimation

In the first part of this project, we will estimate the parameters in the SETAR(2,1,1) model, which we simulated in part 1 of Computer Exercise 1. Recall that the model can be expressed as

$$X_t = \begin{cases} a_{10} + a_{11}X_{t-1} + \epsilon_t & \text{if } X_{t-1} \leq 0 \\ a_{20} + a_{21}X_{t-1} + \epsilon_t & \text{if } X_{t-1} > 0 \end{cases}$$

To keep the following sections feasible, we will estimate two parameters, a_{11} and a_{21}, while keeping the intercepts fixed at $a_{10} = a_{20} = 0$.

To preform the estimation, we start by simulating n realizations from the model where

$$a_{11} = 0.5 \ , \ a_{21} = -0.5 \text{ and } \epsilon_t \sim N(0, 1)$$

To estimate the parameters we use the prediction error method. This is done by defining a lossfunction which has to be minimized. The lossfunction is given by

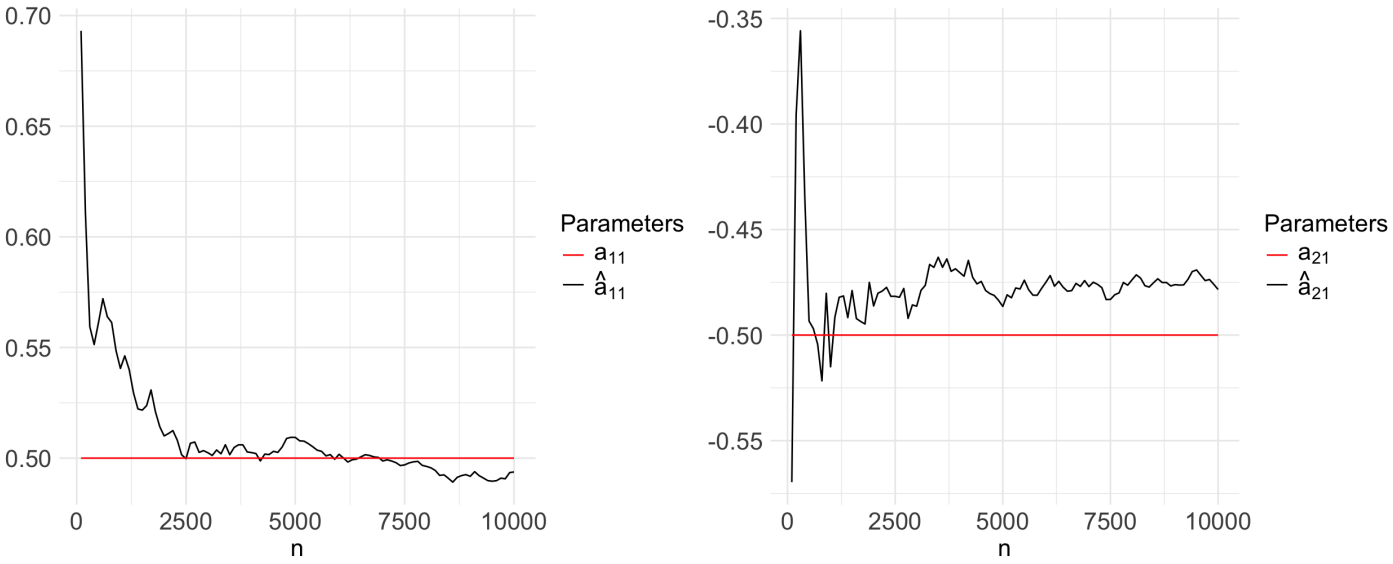
$$Q_N(\theta) = \sum_{i=1}^{N} (x_i - E_\theta[x_i|x_{i-1}])^2 \tag{1}$$

where $\theta = (\tilde{a}_{11}, \tilde{a}_{21})$ represents the candidate parameters for the true model. Further $E_\theta[x_i|x_{i-1}] = \tilde{a}_{11}x_{i-1}$ if $x_{i-1} \leq 0$ else $E_\theta[x_i|x_{i-1}] = \tilde{a}_{21}x_{i-1}$. In theory the parameter estimates are given by the solution to the equations

$$\frac{\partial Q_N}{\partial \tilde{a}_{11}} = 0 \text{ and } \frac{\partial Q_N}{\partial \tilde{a}_{21}} = 0.$$

To minimize the loss function in practice we use the BFGS optimization algorithm. This approach involves performing a Taylor expansion of the loss function $Q_N(\theta)$ around an initial point, θ_0, to approximate the minimum. The method uses both the gradient and an approximation of the Hessian matrix to iteratively find the optimal parameter values. Further does BFGS ensures that the Hessian approximation remains positive definite, which helps maintain stability and ensures convergence to a local minimum. We use the `optim` function in R with the BFGS algorithm

With a fixed seed and an initial parameter guess of $\theta = (1, -1)$, we estimated the parameters for various sample sizes, ranging from $n = 100$ to $n = 10.000$. The purpose of using a fixed seed is to ensure that changes in parameter estimates are only due to the variation in n and not random fluctuations. Figure 1 displays the estimated values of a_{11} and a_{21} for different values of n. The estimation of a_{11}(left plot) demonstrates a clear trend towards the true value as n increases. In contrast, the estimation for a_{21}(right plot) stabilizes at a value slightly offset from the true parameter (approximately 0.05 away) even as n grows. A simple analysis of the distribution of observations in each regime reveals a consistent imbalance, with on average 66% of the simulated observations falling in regime 1 and 33% in regime 2. This imbalance results in a difference in precision for the parameter estimation in each regime. This finding suggest that a much larger sample size may be required to achieve similar convergence for $\hat{a}_{21}$ as for $\hat{a}_{11}$. Table 1 provides specific parameter estimates for three different values of n.



(a) Estimation of a_{11} for different n

(b) Estimation of a_{21} for different n

Figure 1: Estimation of a_{11}(left) and a_{21}(right)

n	$\hat{a}_{11}$	$\hat{a}_{21}$
100	0.69	-0.56
1000	0.54	-0.52
10000	0.49	-0.48

Table 1: Parameter estimations for 3 different n

2 Part 2 - Contour Plots for the Loss Function

In the previous section, we estimated two parameters of a $SETAR(1,1,2)$ model and discussed how sample size affects the precision of these estimates. In this section, we examine this further by visualizing contour plots of the loss function for different subsets of a simulated time series. These contour plots are presented in Figure 2.

The figure shows contour plots of the loss function $Q_n(\theta)$, as defined in 1, across five different subsets of the simulated series with 3000 observations. The first three subsets decrease in sample size, while the last two maintain small sample sizes but include a burn-in period. Table 2 lists the distances between the estimated and true parameter values.

Overall, the contour plots display a somewhat vertically oval shape across all subsets, suggesting that the loss function is steeper in the a_{11} direction than in the a_{21} direction. This is expected, as there is an imbalance in the number of observations in each regime, as discussed previously.

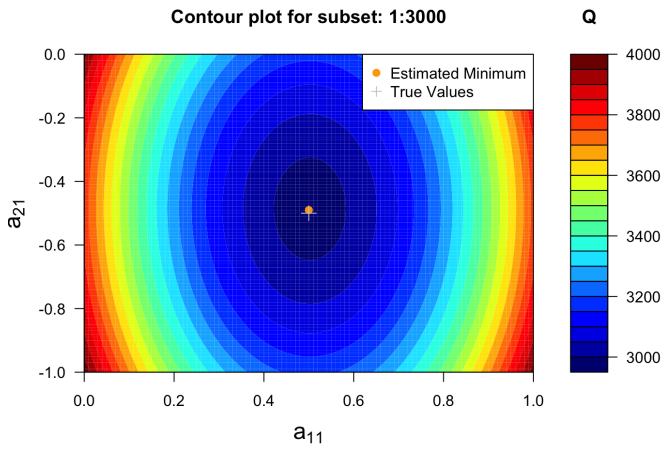
Figure 2(a) illustrates the contour plot for the full set of 3000 observations. Here, the estimated minimum aligns closely with the true parameter values, with a distance of only 0.01 between the estimated and true parameters. The contours are tightly concentrated, and the loss function shows a sharp, well-defined minimum, indicating high confidence in the parameter estimates when the full dataset is used.

For the smaller subsets, shown in 2(b) and 2(c), which represent subsets of 300 and 30 observations, respectively, we see notable changes. In Figure 2(c), the estimated parameter values deviate more from the true values, with a distance of 0.50. The contours are broader, particularly along the a_{21} direction, suggesting a flatter loss function and increased uncertainty in the estimates.

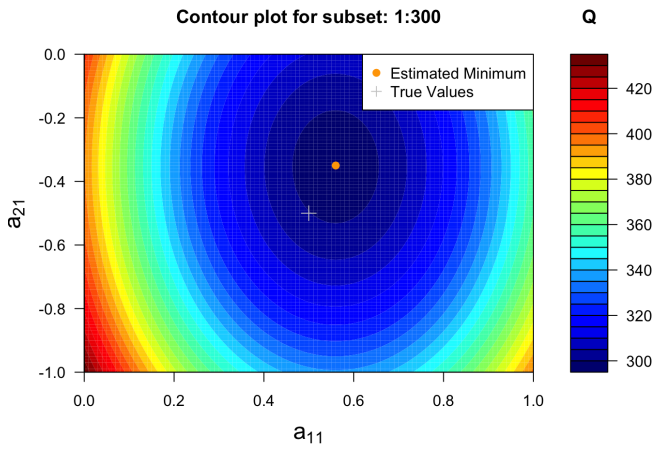
Comparing Figures 2(b) and 2(c) with Figures 2(d) and 2(e), we observe better estimates in the latter, despite the subsets containing the same number of observations. This improvement may be due to how the initial observations in each subset are defined. For (b) and (c), $X_1 = \epsilon_1$, which does not accurately reflect the model. However, in (d) and (e), a burn-in period precedes the definition of the initial value for each subset. Discarding initial observations helps reduce bias from initial conditions and should therefore be applied when estimating parameters from a small sample size.

Simulation	**Error**
1:3000	0.01
1:300	0.16
1:30	0.5
1001:1300	0.13
1001:1030	0.15

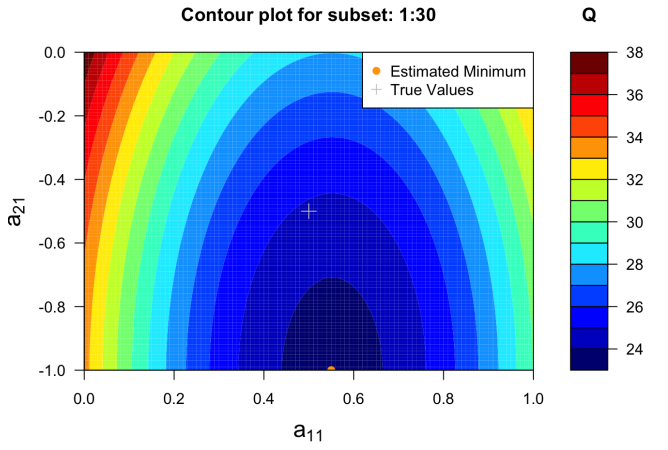
Table 2: Euclidean distance between the estimated and true parameters.



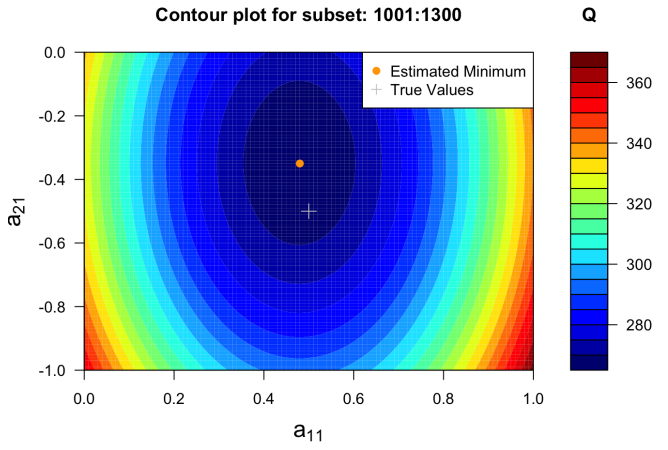
(a) Caption for Figure 1



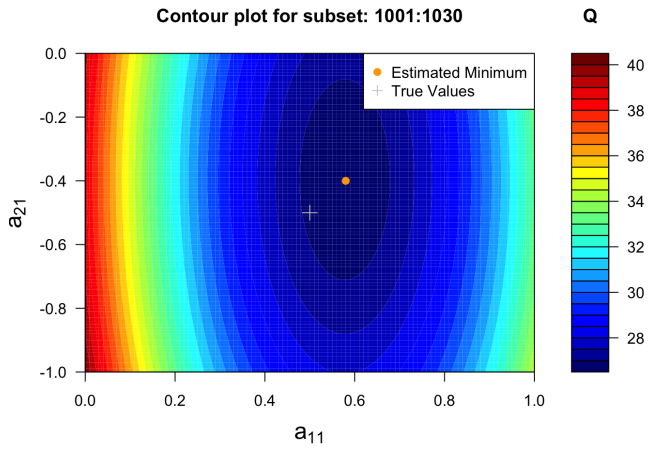
(b) Caption for Figure 2



(c) Caption for Figure 3



(d) Caption for Figure 4



(e) Caption for Figure 5

Figure 2: Overall caption for all figures

3 Part 3 - Simulation from a Doubly Stochastic System

In this section, we will examine a doubly stochastic system. A doubly stochastic system involves two layers of randomness. The first layer relates to the distribution of the observations, represented by one or more parameters. The second layer involves the distribution of one of these parameters, which itself affects the distribution of the observations.

The system we will consider is a non-linear $AR(1) - AR(3)$ model

$$\begin{aligned} Y_t &= \Phi_t X_{t-1} + \epsilon_t \\ \Phi_t - \mu &= \alpha_1(\Phi_{t-1} - \mu) + \alpha_2(\Phi_{t-2} - \mu) + \alpha_3(\Phi_{t-3} - \mu) + \xi_t. \end{aligned}$$

where $\sigma_\epsilon = 1$ and $\sigma_\xi = 0.03$. For the parameter process to be stationary, we set $\alpha_1 = 0.3$, $\alpha_2 = 0.4$ and $\alpha_3 = 0.2$. Note that the roots of the polynomial

$$1 - 0.3z - 0.4z^2 - 0.2z^2 = 0$$

are all outside the unit circle. To encourage the autoregressive coefficient of the observation process to remain within the interval $(-1, 1)$ most of the time (so the observation process is somewhat stable), we have set $\mu = 0.5$. The choice of the low variance of ξ is based on the same reason.

With an alternative parameterization, the model can be reformulated as a linear state-space model. We can write the model for the parameter variation as:

$$\Phi_t = \alpha_1\Phi_{t-1} + \alpha_2\Phi_{t-2} + \alpha_3\Phi_{t-3} + \mu(1 - \alpha_1 - \alpha_2 - \alpha_3).$$

Now by setting $\delta = \mu(1 - \alpha_1 - \alpha_2 - \alpha_3)$ the doubly stochastic system can be described by the linear state space model

$$\begin{pmatrix} \Phi_t \\ \Phi_{t-1} \\ \Phi_{t-2} \\ \delta_t \end{pmatrix} = \begin{pmatrix} \alpha_1 & \alpha_2 & \alpha_3 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \Phi_{t-1} \\ \Phi_{t-2} \\ \Phi_{t-3} \\ \delta_{t-1} \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \xi_t$$

$$Y_t = \begin{pmatrix} Y_{t-1} & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \Phi_t \\ \Phi_{t-1} \\ \Phi_{t-2} \\ \delta_{t-1} \end{pmatrix} + \epsilon_t.$$

The benefit of expressing a doubly stochastic model in a linear state-space form is that it allows for the use of maximum likelihood methods for parameter estimation.

A simulation of the considered process is shown in Figure 3. The top panel displays the observed time series Y_t, while the bottom panel shows the time-varying coefficient Φ_t. Note that the Y_t has larger values or occasional spikes whenever there is a period with $\Phi_t \geq 1$. In other words, high values of the observation process occur during periods when the process is unstable due to too the large autoregressive coefficients.

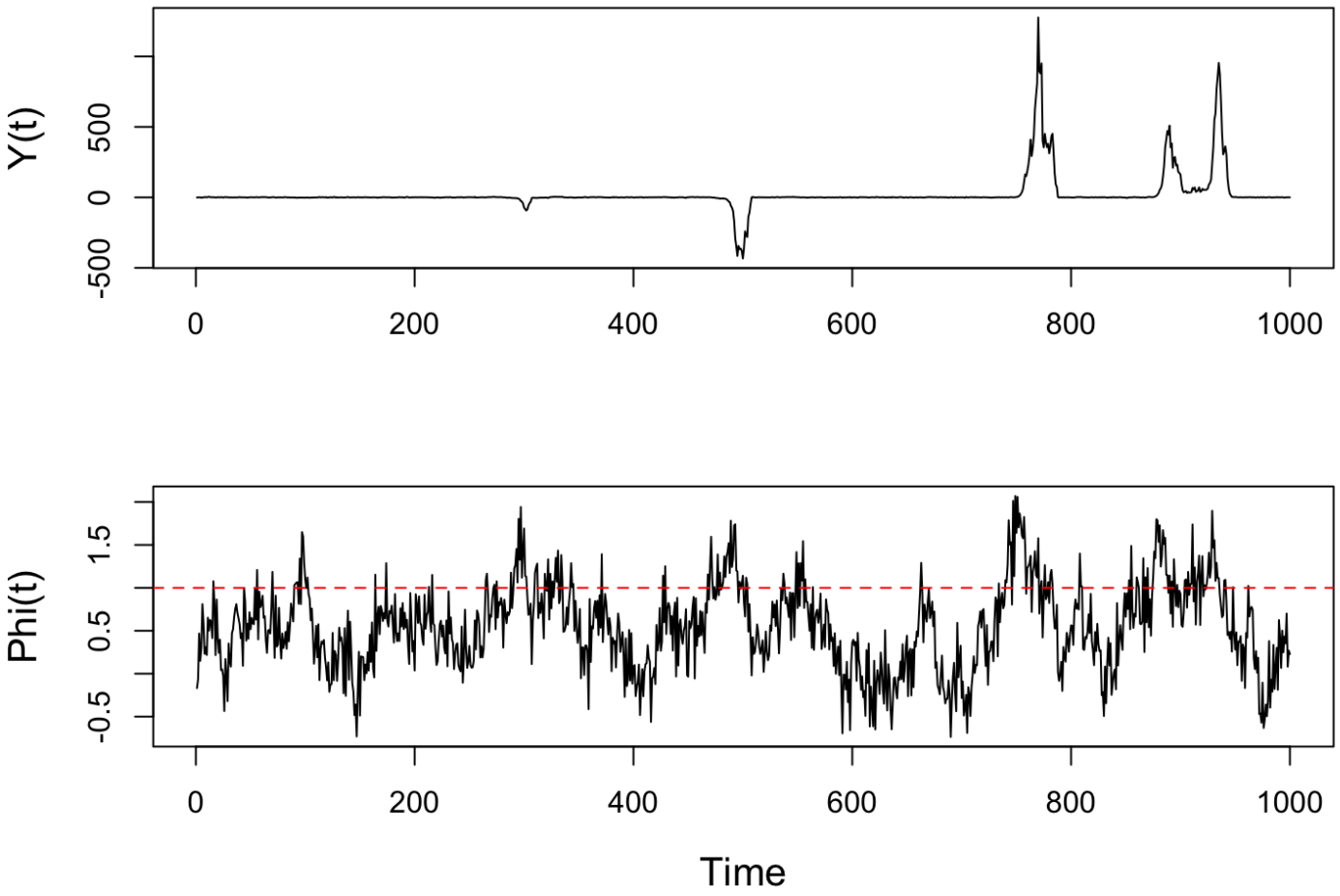


Figure 3: Simulation of a doubly stochastic $AR(1) - AR(3)$ process

4 Part 4 - The Exteded Kalman Filter

We consider the following simple state space model:

$$x_{t+1} = ax_t + v_t \tag{2}$$
$$y_t = x_t + e_t \tag{3}$$

where a is an unknown parameter and v_t and e_t are mutually uncorrelated white noise processes with variances σ_v^2 and σ_e^2 respectively.

4.1 Part 4a - Simulating and rewriting the model

We perform 20 simulations of the model in R, where $a = 0.4$ and the variances of the noise processes $\sigma_v^2 = \sigma_e^2 = 1$. The R code for the simulation is shown in the appendix.

In part 4b, we wish to estimate the parameter a, using the Extended Kalman Filter (EKF). In order to do this we first need to rewrite the model by including the parameter a in the state vector. We follow the method in section 7.4 from the Book (2) and get the new state vector:

$$Z_t = \begin{pmatrix} x_t \\ a_t \end{pmatrix}$$

We assume $a_{t+1} = a_t + 0 = a_t$, as it is a constant, which then leads to the following non-linear state space model:

$$Z_{t+1} = \begin{pmatrix} x_{t+1} \\ a_{t+1} \end{pmatrix} = \begin{pmatrix} a_t x_t + v_t \\ a_t \end{pmatrix} \tag{4}$$

$$y_t = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} x_t \\ a_t \end{pmatrix} + e_t \tag{5}$$

4.2 Part 4b - Convergence properties of the EKF

We use the EKF to estimate the parameter a in the 20 different simulations. We start by doing choosing and initial value of $a = 0.5$ and afterwards. For each initial value of a, we test four different combinations of σ_v^2 and the initial value of the variance parameter a ($InitVar_a$). For the R implementation of the EKF we have taken inspiration from `EKF-Example_new.R`. The plots for $a = 0.5$ are shown below in Figure 4 and plots for $a = -0.5$ are shown in Figure 5

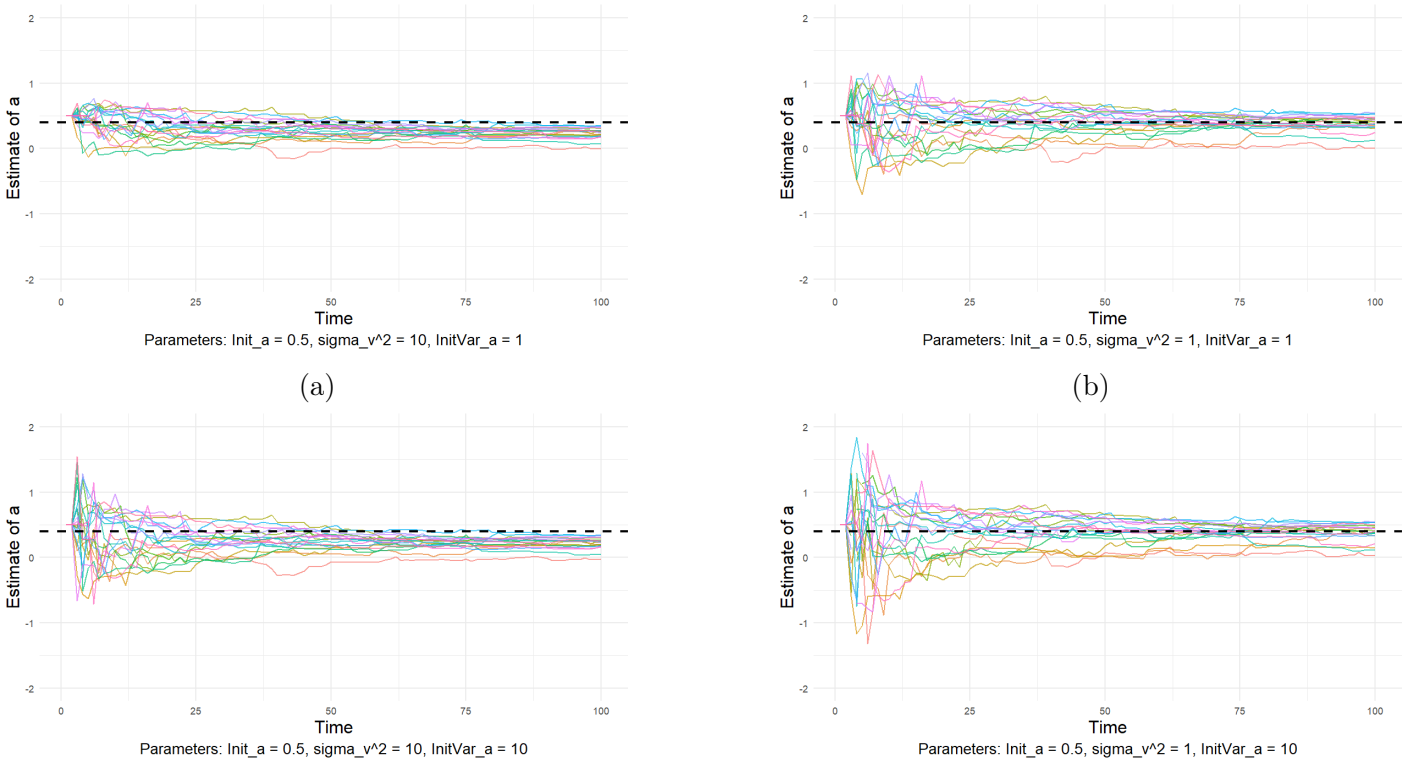


Figure 4: EFK Estimates of Parameter Over Time with initial guess of $a = 0.5$

We now wish to analyze how the different parameters effect the EKF estimates and their convergence properties.

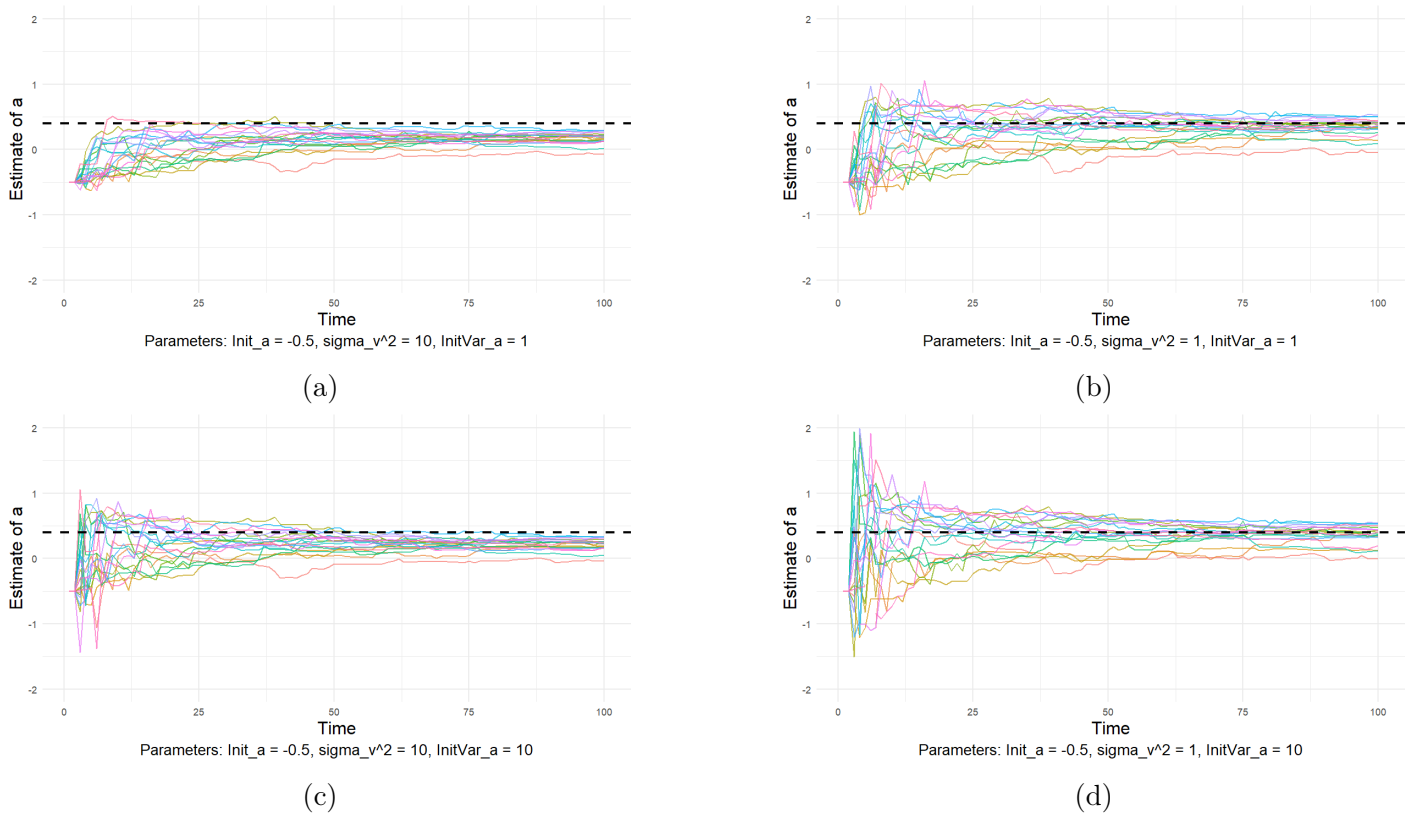


Figure 5: EFK Estimates of Parameter Over Time with initial guess of $a = -0.5$

4.1 Effect of σ_v^2

σ_v^2 is the process noise and it determines how much the EKF relies on the model. If σ_v^2 is high, the filter will rely more on the observations than the model predictions, which means it expects the systems changes and behavior to be more unpredictable. Whereas if σ_v^2 is low the EKF has confidence in the model, assumes that the system will have more predictable changes and thus relies more on the predictions rather than observations.

We start by looking at the cases with $\sigma_v^2 = 10$ in Figure 4a, 4c, 5a and 5c. We notice, that there seems to be a consistent underestimation of the true parameter a in all cases. We remember that the true process noise is $\sigma_v^2 = 1$, so $\sigma_v^2 = 10$ is a pretty high overestimation of this value. When we assume a higher process noise variance, the EKF sees the differences between the predicted and observed values as normal system noise, instead of as a signal to adjust the parameter a. This effect of σ_v^2 seems to align with the theory provided in the Article about the asymptotic behavior of the EKF(1). The article notes that an overestimation of the process noise makes the EKF less responsive to parameter adjustments leading to systematic biases.

We further notice, that a higher σ_v^2 seems to lead to faster convergence of the parameter estimates. Although, they converge to a biased estimated a value.

The cases in Figure 4d, 4b, 5d and 5b, have $\sigma_v^2 = 1$, which is the true process noise of the model. When looking at these plots, we do not see the same underestimation. We observe more initial fluctuations, and slower, but more accurate convergence around the true value of a.

4.2 Effect of initial value of parameter a

We note that the true value of $a = 0.4$ and we are here comparing two initial values of a, namely $a_{init} = 0.5$ and $a_{init} = -0.5$. Where one of them is a fairly good initial guess of the true value, while the other one is an inaccurate guess of a.

In the cases where we have $a_{init} = 0.5$, that are shown in Figure 4 we can see that the estimates start closer to the true value of a. In the cases where $a_{init} = -0.5$ that are shown in Figure 5, all the initial estimates start below the true value of a and needs to make upwards adjustments before "catching up" to the cases that start closer to the true value.

It appears that the initial a mainly determines, where the EKF begins adjusting the value of a, In the beginning we notice a slight underestimation bias in Figure 5a and 5b, but it doesn't seem to impact the end result, as all cases in Figure 5 seem to approximately converge to the same values as the corresponding parameter combination in Figure 4.

We further notice that in the case in Figure 5a where we have an initial underestimation due to the initial value of a, as well as a convergence underestimation due to the process noise, combined with the low value of $InitVar_a$ leads to a constant underestimation where only 2-3 of the simulations even cross the true value at any time.

4.3 Effect of the initial value of the variance for parameter a

The value of the initial variance of a says something about our confidence in the initial guess of a. An initial variance like $initVar_a = 1$ means, that we are highly confident in the initial guess of a, which makes the EKF less flexible, when it comes to adjusting the value of a. If we set the variance to a lower value, it makes the EKF estimates converge faster. If the initial guess is bad, then the lack of flexibility makes it, so that it takes longer to fix the initial error and can also lead to lasting bias.

A higher initial variance of a like $InitVar_a = 10$ means that we are not that confident in our value of a and thus it gives the EKF more flexibility when it comes to adjusting the initial value in the beginning. This helps the filter account for the differences between the model predictions and the observations. Looking at the plots, it is clearly seen when comparing cases of $InitVar_a = 1$ to $InitVar_a = 10$, that $InitVar_a = 10$ leads to much higher oscillations in the earlier time steps before converging.

In our cases the difference in convergence accuracy between the cases isn't so clear, which might be due to $a = -0.5$ not being a "bad enough guess", but we would expect the EKF to perform better for a bad initial guess of a, if the $InitVar_a$ is high value, while if the guess is accurate we would expect the EKF to perform better for a small value of $InitVar_a$.

5 References

References

[1] Ljung L., Asymptotic Behavior of the Extended Kalman Filter as a Parameter Estimator for Linear Systems [1979].

[2] Madsen H., Modeling Non-Linear and Non-stationary Time Series [2006].

6 Appendex A

The code used for solving each part is given here:

Part 1

```
################################################################################
# Part 1 - prediction error method for parameter estimation
################################################################################

rm(list = ls())
dev.off()



# Define the range of n values (e.g., 30, 40, 50, ..., 30000)
n_values <- seq(100, 10000, by = 100)

# Set true parameters
a11_true <- 0.5
a21_true <- -0.5

# Initialize a data frame to store results
results <- data.frame(n = integer(), a11_estimate = numeric(), a21_estimate = numeric())

# Function to simulate the STAR model
simulate_STAR <- function(n, a11_true, a21_true) {
  set.seed(1234)
  X <- numeric(n)
  eps <- rnorm(n)

  # Initial value for X
  X[1] <- rnorm(1)

  # Simulate the STAR process
  for (t in 2:n) {
    if (X[t-1] <= 0) {
      X[t] <- a11_true * X[t-1] + eps[t]
    } else {
      X[t] <- a21_true * X[t-1] + eps[t]
    }
  }

  return(X)
}

# Define the loss function
RSSSetar <- function(theta, X) {
  a11 <- theta[1]
  a21 <- theta[2]
  n <- length(X)
```

```
  # Compute prediction errors
  prediction_errors <- numeric(n - 1)

  for (t in 2:n) {
    if (X[t-1] <= 0) {
      prediction_errors[t] <- X[t] - a11 * X[t-1]
    } else {
      prediction_errors[t] <- X[t] - a21 * X[t-1]
    }
  }

  # Return the sum of squared errors
  sum(prediction_errors^2)
}

# Loop over each n value to estimate parameters
for (n in n_values) {
  # Simulate STAR model data
  X <- simulate_STAR(n, a11_true, a21_true)

  # Estimate parameters using optim
  theta_hat <- optim(par = c(3, -3), fn = RSSSetar, X = X, method = "BFGS")$par

  # Store results in the data frame
  results <- results %>%
    add_row(n = n, a11_estimate = theta_hat[1], a21_estimate = theta_hat[2])
}

results

# plot estimation of a11
ggplot(data = results, aes(x = n)) +
  geom_line(aes(y = a11_estimate, color = "a11_hat")) +
  geom_line(aes(y = 0.5, color = "a11"), linetype = "solid") +
  scale_color_manual(
    values = c("a11_hat" = "black", "a11" = "red"),
    labels = c("a11_hat" = expression(hat(a)[11]), "a11" = expression(a[11]))
  ) +
  labs(title = NULL, y = NULL, x = "n", color = "Parameters") +
  theme_minimal() +
  theme(
    axis.title = element_text(size = 16),
    axis.text = element_text(size = 16),
    legend.title = element_text(size = 16),
    legend.text = element_text(size = 16)
  )

# plot estimation of a21
ggplot(data = results, aes(x = n)) +
  geom_line(aes(y = a21_estimate, color = "a21_hat")) +
  geom_line(aes(y = -0.5, color = "a21"), linetype = "solid") +
```

```
  scale_color_manual(
    values = c("a21_hat" = "black", "a21" = "red"),
    labels = c("a21_hat" = expression(hat(a)[21]), "a21" = expression(a[21]))
  ) +
  labs(title = NULL, y = NULL, x = "n", color = "Parameters") +
  theme_minimal() +
  theme(
    axis.title = element_text(size = 16),
    axis.text = element_text(size = 16),
    legend.title = element_text(size = 16),
    legend.text = element_text(size = 16)
  )
```

Part 2

```
################################################################################
# Part 2 - Contour plots
################################################################################



rm(list = ls())
dev.off()
set.seed(1234)


# Simulate the STAR model process ----------------------------------------------
n <- 3000 # number of observations
X <- numeric(n) # storage for X_t
eps <- rnorm(n) # random errors from N(0,1)

# Initial values and parameters
X[1] <- rnorm(1)
a11_true <- 0.5
a21_true <- -0.5

# Simulating STAR model
for (t in 2:n) {
  if (X[t-1] <= 0) {
    X[t] <- a11_true * X[t-1] + eps[t]
  } else {
    X[t] <- a21_true * X[t-1] + eps[t]
  }
}

D <- data.frame(X = X[-1], X1 = X[-n])
D_test <- D[1001:1030, c("X", "X1")]
sum(D_test$X1 <= 0)
sum(D_test$X1 > 0)
```

```r
# Define Qn( ) for subsets of different sizes ---------------------------------
RSSSetar <- function(theta, X_subset) {

  a11 <- theta[1] # Parameter for regime 1
  a21 <- theta[2] # Parameter for regime 2

  n <- length(X_subset)
  prediction_errors <- numeric(n - 1)

  for (t in 2:n) {
    if (X_subset[t-1] <= 0) {
      prediction_errors[t] <- X_subset[t] - a11 * X_subset[t-1]
    } else {
      prediction_errors[t] <- X_subset[t] - a21 * X_subset[t-1]
    }
  }

  # Return the sum of squared prediction errors
  sum(prediction_errors^2)
}


# range for a_11 and a_21
a_11 <- seq(0, 1, 0.01) # Range for a11
a_21 <- seq(-1, 0, 0.01) # Range for a21


# Define grid search for RSSSetar function for all a_11 and a_21 values
RSSSetar_grid <- function(X_subset) {
  grid <- matrix(NA, nrow = length(a_11), ncol = length(a_21))

  for (i in 1:length(a_11)) {
    for (j in 1:length(a_21)) {
      # Theta with a_11, a_21 as grid points
      theta <- c(a_11[i], a_21[j])
      grid[i, j] <- RSSSetar(theta, X_subset)
    }
  }
  return(grid)
}

# Create subsets of data
subsets <- list(
  "1:3000" = X[1:3000],
  "1:300" = X[1:300],
  "1:30" = X[1:30],
  "1001:1300" = X[1001:1300],
  "1001:1030" = X[1001:1030]
)

# Plot for first subset -------------------------------------------------------
```

```r
X_subset <- subsets[[names(subsets)[1]]]
# Calculate the loss function grid for this subset
Q_grid <- RSSSetar_grid(X_subset)


## find values of a11 and a21 which minimizes loss function

# find
min_index <- which(Q_grid == min(Q_grid), arr.ind = TRUE)

# Extract the indices for a_11 and a_21 that correspond to the minimum value
min_a11_index <- min_index[1]
min_a21_index <- min_index[2]

# Find the corresponding a_11 and a_21 values
min_a11 <- a_11[min_a11_index]
min_a21 <- a_21[min_a21_index]
min_a11
min_a21



filled.contour(
  a_11, a_21, Q_grid,
  color.palette = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
      "yellow", "#FF7F00", "red", "#7F0000")),
  plot.title = title(main = paste("Contour plot for subset:", names(subsets)[1]),
                     xlab = expression(a[11]), ylab = expression(a[21]),
                     cex.lab = 1.5),
  key.title = title(main = "Q"),
  plot.axes = {
    # Redraw the x and y axes
    axis(1) # x-axis
    axis(2) # y-axis

    # Add points for (min_a11, min_a21) and (0.5, -0.5)
    points(min_a11, min_a21, pch = 19, col = "orange",cex = 1) # Minimum point
    points(0.5, -0.5, pch = 3, col = "gray80", cex = 1.5)  # True parameter values

    # Add a legend
    legend("topright", legend = c("Estimated Minimum", "True Values"),
           col = c("orange", "gray80"), pch = c(19,3))
  }
)

# Euclidian distance
dist_1_3000 <- sqrt((min_a11-0.5)^2 + (min_a21 + 0.5)^2)



# Plot for second subset ------------------------------------------------------
X_subset <- subsets[[names(subsets)[2]]]
```

```r
# Calculate the loss function grid for this subset
Q_grid <- RSSSetar_grid(X_subset)


## find values of a11 and a21 which minimizes loss function

# find
min_index <- which(Q_grid == min(Q_grid), arr.ind = TRUE)

# Extract the indices for a_11 and a_21 that correspond to the minimum value
min_a11_index <- min_index[1]
min_a21_index <- min_index[2]

# Find the corresponding a_11 and a_21 values
min_a11 <- a_11[min_a11_index]
min_a21 <- a_21[min_a21_index]
min_a11
min_a21



filled.contour(
  a_11, a_21, Q_grid,
  color.palette = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
      "yellow", "#FF7F00", "red", "#7F0000")),
  plot.title = title(main = paste("Contour plot for subset:", names(subsets)[2]),
                     xlab = expression(a[11]), ylab = expression(a[21]),
                     cex.lab = 1.5),
  key.title = title(main = "Q"),
  plot.axes = {
    # Redraw the x and y axes
    axis(1) # x-axis
    axis(2) # y-axis

    # Add points for (min_a11, min_a21) and (0.5, -0.5)
    points(min_a11, min_a21, pch = 19, col = "orange",cex = 1) # Minimum point
    points(0.5, -0.5, pch = 3, col = "gray80", cex = 1.5)  # True parameter values

    # Add a legend
    legend("topright", legend = c("Estimated Minimum", "True Values"),
           col = c("orange", "gray80"), pch = c(19,3))
  }
)


# Euclidian distance
dist_1_300 <- sqrt((min_a11-0.5)^2 + (min_a21 + 0.5)^2)


# Plot for thirth subset ------------------------------------------------------
X_subset <- subsets[[names(subsets)[3]]]
# Calculate the loss function grid for this subset
```

```r
Q_grid <- RSSSetar_grid(X_subset)


## find values of a11 and a21 which minimizes loss function

# find
min_index <- which(Q_grid == min(Q_grid), arr.ind = TRUE)

# Extract the indices for a_11 and a_21 that correspond to the minimum value
min_a11_index <- min_index[1]
min_a21_index <- min_index[2]

# Find the corresponding a_11 and a_21 values
min_a11 <- a_11[min_a11_index]
min_a21 <- a_21[min_a21_index]
min_a11
min_a21



filled.contour(
  a_11, a_21, Q_grid,
  color.palette = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
      "yellow", "#FF7F00", "red", "#7F0000")),
  plot.title = title(main = paste("Contour plot for subset:", names(subsets)[3]),
                     xlab = expression(a[11]), ylab = expression(a[21]),
                     cex.lab = 1.5),
  key.title = title(main = "Q"),
  plot.axes = {
    # Redraw the x and y axes
    axis(1) # x-axis
    axis(2) # y-axis

    # Add points for (min_a11, min_a21) and (0.5, -0.5)
    points(min_a11, min_a21, pch = 19, col = "orange",cex = 1) # Minimum point
    points(0.5, -0.5, pch = 3, col = "gray80", cex = 1.5)  # True parameter values

    # Add a legend
    legend("topright", legend = c("Estimated Minimum", "True Values"),
           col = c("orange", "gray80"), pch = c(19,3))
  }
)

# Euclidian distance
dist_1_30 <- sqrt((min_a11-0.5)^2 + (min_a21 + 0.5)^2)


# Plot for 4th subset ------------------------------------------------------
X_subset <- subsets[[names(subsets)[4]]]
# Calculate the loss function grid for this subset
Q_grid <- RSSSetar_grid(X_subset)
```

```r
## find values of a11 and a21 which minimizes loss function

# find
min_index <- which(Q_grid == min(Q_grid), arr.ind = TRUE)

# Extract the indices for a_11 and a_21 that correspond to the minimum value
min_a11_index <- min_index[1]
min_a21_index <- min_index[2]

# Find the corresponding a_11 and a_21 values
min_a11 <- a_11[min_a11_index]
min_a21 <- a_21[min_a21_index]
min_a11
min_a21



filled.contour(
  a_11, a_21, Q_grid,
  color.palette = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
      "yellow", "#FF7F00", "red", "#7F0000")),
  plot.title = title(main = paste("Contour plot for subset:", names(subsets)[4]),
                     xlab = expression(a[11]), ylab = expression(a[21]),
                     cex.lab = 1.5),
  key.title = title(main = "Q"),
  plot.axes = {
    # Redraw the x and y axes
    axis(1) # x-axis
    axis(2) # y-axis

    # Add points for (min_a11, min_a21) and (0.5, -0.5)
    points(min_a11, min_a21, pch = 19, col = "orange",cex = 1) # Minimum point
    points(0.5, -0.5, pch = 3, col = "gray80", cex = 1.5)  # True parameter values

    # Add a legend
    legend("topright", legend = c("Estimated Minimum", "True Values"),
           col = c("orange", "gray80"), pch = c(19,3))
  }
)


# Euclidian distance
dist_1001_1300 <- sqrt((min_a11-0.5)^2 + (min_a21 + 0.5)^2)



# Plot for 5th subset ------------------------------------------------------
X_subset <- subsets[[names(subsets)[5]]]
D <- data.frame(X = X[-1], X1 = X[-n])
D_test <- D[1:30, c("X", "X1")]
sum(D_test$X1 <= 0)
```

```r
sum(D_test$X1 > 0)


# Calculate the loss function grid for this subset
Q_grid <- RSSSetar_grid(X_subset)


## find values of a11 and a21 which minimizes loss function

# find
min_index <- which(Q_grid == min(Q_grid), arr.ind = TRUE)

# Extract the indices for a_11 and a_21 that correspond to the minimum value
min_a11_index <- min_index[1]
min_a21_index <- min_index[2]

# Find the corresponding a_11 and a_21 values
min_a11 <- a_11[min_a11_index]
min_a21 <- a_21[min_a21_index]
min_a11
min_a21



filled.contour(
  a_11, a_21, Q_grid,
  color.palette = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
      "yellow", "#FF7F00", "red", "#7F0000")),
  plot.title = title(main = paste("Contour plot for subset:", names(subsets)[5]),
                     xlab = expression(a[11]), ylab = expression(a[21]),
                     cex.lab = 1.5),
  key.title = title(main = "Q"),
  plot.axes = {
    # Redraw the x and y axes
    axis(1) # x-axis
    axis(2) # y-axis

    # Add points for (min_a11, min_a21) and (0.5, -0.5)
    points(min_a11, min_a21, pch = 19, col = "orange",cex = 1) # Minimum point
    points(0.5, -0.5, pch = 3, col = "gray80", cex = 1.5)  # True parameter values

    # Add a legend
    legend("topright", legend = c("Estimated Minimum", "True Values"),
           col = c("orange", "gray80"), pch = c(19,3))
  }
)
# Euclidian distance
dist_1001_1030 <- sqrt((min_a11-0.5)^2 + (min_a21 + 0.5)^2)
```

Part 3

```
################################################################################
# Part 3 - doubly stochastic system
################################################################################


rm(list = ls())
set.seed(123)


n <- 1000
X <- numeric(n)
Phi_t <- numeric(n)


phi1 <- 0.3
phi2 <- 0.4
phi3 <- 0.2
mu <- 0.5
sigma_ksi <- 0.3
sigma_eps <- 1
ksi <- rnorm(n, sd = sigma_ksi)
eps <- rnorm(n, sd = sigma_eps)
Phi_t[1] <- ksi[1]
Phi_t[2] <- ksi[2]
Phi_t[3] <- ksi[3]
X[1] <- eps[1]


for (t in 4:n){
  Phi_t[t] <- phi1*(Phi_t[t-1] - mu) +
              phi2*(Phi_t[t-2] - mu) +
              phi3*(Phi_t[t-3] - mu) + mu + ksi[t]

  X[t] <- Phi_t[t]*X[t-1] + eps[t]
}

df <- data_frame(X, Phi_t, t = seq(1, length(X)))

par(mfrow = c(2, 1), mar = c(4, 4, 2, 1), oma = c(0, 0, 0, 0))

plot(X, type = "l",xlab = " " , ylab = "Y(t)", cex.lab = 1.3)
plot(Phi_t, type = "l", xlab = "Time", ylab = "Phi(t)", cex.lab = 1.3)
abline(h = 1, col = "red", lty = 2)
```

Part 4

```
# Load libraries
library(ggplot2)
library(reshape2)
```

```r
rm(list = ls())
set.seed(40)

################################
# Part 4a - Simulation
################################

# Simulation parameters
T <- 100        # How many time steps
a <- 0.4        # True parameter
sigma_v_sim <- 1 # Process noise standard deviation in simulation
sigma_e <- 1    # Measurement noise standard deviation for simulation
n <- 20      # Number of simulations

#Store simulations in list
simulations <- list()

#Perform n simulations
for (sim in 1:n) {

  # Initialize state and observation vectors
  x <- numeric(T)
  y <- numeric(T)

  # Initial state
  x[1] <- 0

  # Simulate the system
  for (t in 2:T) {
    x[t] <- a * x[t-1] + rnorm(1, mean = 0, sd = sigma_v_sim)
  }

  # Simulate observations
  y <- x + rnorm(T, mean = 0, sd = sigma_e)

  # Store the simulations in list
  simulations[[sim]] <- list(x = x, y = y)
}

#############################################################
# Part 4b - Extended Kalman Filter
#############################################################

#-------------------------------------------------
# Define Kalman filter
#-------------------------------------------------


#We define the Extended Kalman Filter as a function. We have taken
#inspiration from the EKF-Example_new.R.
```

```r
#In the function:
# y are the observations
# zt_init is Initial state vector with a guess of 'a'
# Pt_init is the Initial state covariance
# Rv is process noise covariance matrix for the EKF
# Re is Measurement noise variance in the EKF
EKF <- function(y, zt_init, Pt_init, Rv, Re) {
  T <- length(y)
  zt <- zt_init
  Pt <- Pt_init
  Ht <- t(c(1, 0))

  Z <- matrix(NA, nrow = T, ncol = 2)
  Z[1, ] <- zt

  for (t in 1:(T - 1)) {
    # Jacobian Matrix Ft based on current state
    Ft <- matrix(c(zt[2], 0, zt[1], 1), ncol = 2)

    # Prediction step
    zt <- c(zt[2] * zt[1], zt[2]) # Predicted state
    Pt <- Ft %*% Pt %*% t(Ft) + Rv # Predicted state covariance

    # Update step
    res <- y[t] - zt[1] # Innovation (residual)
    St <- Ht %*% Pt %*% t(Ht) + Re # Innovation covariance
    Kt <- Pt %*% t(Ht) %*% solve(St) # Kalman gain
    zt <- zt + Kt * res # Updated state estimate
    Pt <- (diag(2) - Kt %*% Ht) %*% Pt # Updated state covariance

    # Store results
    Z[t + 1, ] <- zt # Store state
  }

  return(Z)
}


#-------------------------------------------
# Prepare the different parameter combinations
#-------------------------------------------

#We define the combination of EKF parameters given in the assignment description
params_EKF <- data.frame(
  sigma_v2_EKF = c(10, 1, 10, 1, 10, 1, 10, 1),
  aVarInit = c(1, 1, 10, 10, 1, 1, 10, 10),
  aInit = c(0.5, 0.5, 0.5, 0.5, -0.5, -0.5, -0.5, -0.5),
  stringsAsFactors = FALSE
)

# Caption for the plots
```

```
params_EKF$label <- paste0(
  "Init_a = ", params_EKF$aInit,
  ", sigma_v^2 = ", params_EKF$sigma_v2_EKF,
  ", InitVar_a = ", params_EKF$aVarInit
)

#----------------------------------------
# Use EKF on all simulations
#----------------------------------------

# Store results in list (for plotting)
plot_data_list <- list()

# Loop over parameter combinations
for (i in 1:nrow(params_EKF)) {
  # Extract parameters
  sigma_v2_EKF <- params_EKF$sigma_v2_EKF[i]
  sigma_v_EKF <- sqrt(sigma_v2_EKF)
  aVarInit <- params_EKF$aVarInit[i]
  aInit <- params_EKF$aInit[i]
  label <- params_EKF$label[i]

  # EKF Parameters for each case
  Rv <- matrix(c(sigma_v2_EKF, 0, 0, 0), ncol = 2) # Process noise covariance matrix for the
      EKF
  Re <- 1 # Measurement noise variance in the EKF

  # List to store the estimates of a for all simulations of parameter combination
  a_esti <- list()

  # Loop over simulations
  for (sim in 1:n) {
    y <- simulations[[sim]]$y

    # Initialize EKF
    zt_init <- c(0, aInit) # Initial state vector with a guess of 'a'
    Pt_init <- matrix(c(Re, 0, 0, aVarInit), nrow = 2, ncol = 2)

    # Run EKF
    Z <- EKF(y, zt_init, Pt_init, Rv, Re)

    # Store the estimates of a over time for each simulation
    a_esti[[sim]] <- Z[, 2]
  }

  # Prepare data for plotting for parameter combinations
  a_esti_df <- do.call(cbind, a_esti)
  colnames(a_esti_df) <- paste0("sim_", 1:n)
  a_esti_df <- as.data.frame(a_esti_df)
  a_esti_df$Time <- 1:T
  a_esti_long <- melt(a_esti_df, id.vars = "Time", variable.name = "Simulation", value.name
      = "a_estimate")
```

```r
  # Add label to the data
  a_esti_long$Parameters <- label

  # Store in plot data list
  plot_data_list[[i]] <- a_esti_long
}

#---------------------------------
# Create Plots of each combination
#---------------------------------

# Combine all data for plotting
plot_data <- do.call(rbind, plot_data_list)

# Plot Parameter Combination Order
plot_data$Parameters <- factor(plot_data$Parameters, levels = unique(plot_data$Parameters))

# We loop over each parameter combination to make one plot for each.
param_labels <- levels(plot_data$Parameters)

for (label in param_labels) {
  data_subset <- subset(plot_data, Parameters == label)

  # Create plots
  plot <- ggplot(data_subset, aes(x = Time, y = a_estimate, group = Simulation)) +
    geom_line(aes(color = Simulation), alpha = 0.7) +
    geom_hline(yintercept = a, linetype = "dashed", color = "black", size = 1) + # True
        value of a
    labs(caption = paste("Parameters:", label),
         x = "Time",
         y = "Estimate of a") +
    ylim(-2,2) +
    theme_minimal() +
    theme(
      legend.position = "none", # Remove legend for simulations
      plot.caption = element_text(size = 14, hjust = 0.5),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16)
    )

  print(plot)

}
```